

$R \leq 10^{10}$ 。此题较难。

7.6 状态压缩 DP

先用一道例题引出状态压缩 DP 的概念。

1. 例题 1

poj 3254 “Corn Fields”

农夫约翰有一片长方形土地，划成 M 行 N 列的方格。他准备种玉米、养牛，不过有些格子很贫瘠，不适合种玉米。还有，牛不喜欢在一起吃，所以牛不能放在相邻的格子里。给出这块地的情况，求约翰有多少个种玉米的方案。所有方格都不种玉米也算一种方案。

输入：第 1 行是 M 和 N , $1 \leq M, N \leq 12$ 。后面有 M 行, 描述方格情况, 1 表示肥沃, 0 表示贫瘠。

输出：方案数, 用 10^9 取模。

输入样例：

2 3

1 1 1

0 1 0

输出样例：

9

提示：在样例中有 9 种方案。

1	2	3
	4	

分别是 $\{\}, \{1\}, \{2\}, \{3\}, \{4\}, \{1,3\}, \{1,4\}, \{3,4\}, \{1,3,4\}$ 。

这个方格图共 $m \times n$ 个格子, 有 $2^{m \times n}$ 种排列, 无法用暴力法计算。

用下面的方法编程计算, 算法复杂度是 $O(m2^n2^n)$ 。

1) 方格的表示

很容易想到, 可以用二进制数来描述方格, 1 表示种玉米, 0 表示不种玉米。在样例中, 第 1 行的 3 个方格都是肥沃的, 排除相邻的情况, 有以下 5 种种玉米的方案:

第 1 行	编 号	1	2	3	4	5
	方 案	000	001	010	100	101

这里的编号并不是多余的, 在下面设计 DP 状态的时候有用。

2) DP 状态和状态转移

如何设计 DP 状态, 把问题从小规模逐步扩展到大规模? 可以按行进行扩展。

上面已经得到了第 1 行的 5 种方案, 下面继续扩展第 2 行。

在样例中, 第 2 行只有两种方案, 即 000、010。

第 2 行	编 号	1	2
	方 案	000	010

如果第 2 行选编号 1 的 000, 第 1 行可以选 5 种方案而不冲突。

如果第 2 行选编号 2 的 010, 与第 1 行的 010 有冲突, 第 1 行的其他 4 种方案没问题。

共 $5+4=9$ 种方案。

用 $dp[i][j]$ 表示第 i 行采用第 j 种编号的方案时前 i 行可以得到的可行方案总数。例如, $dp[2][2]=4$ 表示第 2 行使用第 2 种方案(即 010)时的方案总数是 4。

从第 $i-1$ 行转移到第 i 行, 状态转移方程如下:



$$dp[i][k] = \sum_{j=1}^n dp[i-1][j]$$

其中 j 是第 $i-1$ 行可行方案的编号, 而且所有的 $dp[i-1][j]$ 与第 i 行不冲突

那么 $Path = (v_0 \rightarrow v_1) + (v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_0)$

所以,问题转变为:求经过所有城市的最短回路→从某个城市到起点的最短路径。

DP 状态设计如下:假设已经访问过的城市集合是 S ,当前所在城市是 v ,用 $dp[S][v]$ 表示从 v 出发访问剩余的所有城市最后回到起点的路径费用总和的最小值。状态转移方程如下:

$dp[V][0] = 0$ // V 是最后一个城市

$dp[S][v] = \min(dp[S \cup \{u\}][u] + \text{dist}(v, u) \mid u \notin S)$

城市集合 S 如何表示?这就用到状态压缩 DP 的技巧:把路径“压缩”到一个二进制数中。定义:

```
int dp[1 << MAXN][MAXN];
```

MAXN 是城市数量,当 $MAXN = 15$ 时, $1 << MAXN = 2^{15} = 32768$, $0 \sim 32768$ 内的每个数的二进制表示就是一个可能的路径,二进制数中的 1 表示选中一个城市,0 表示不选中。例如 $S = 000\ 0000\ 0000\ 0101_2$,末尾的 101 表示已访问过城市 2、0。在下面的代码中,“ $dp[s | 1 << u][u]$ ”,其中的 $s | 1 << u$,表示在已访问过的城市集合 S 中加入一个新访问的城市 u 。

下面是部分示意代码^①。

```
int dp[1 << MAXN][MAXN];
void solve(){
    memset(dp, INF, sizeof(dp));           //初始化为无穷大
    dp[(1 << n) - 1][0] = 0;               //从最后一个点出发到起点 0,已经没有城市可
                                           //以走,所以到起点 0 的最小路径费用是 0
    for (int s = (1 << n) - 2; s >= 0; s--) // $O(2^n)$ 
        for (int v = 0; v < n; v++)         // $O(n)$ 
            for (int u = 0; u < n; u++)      // $O(n)$ 
                if (!(s >> u & 1))
                    dp[s][v] = min(dp[s][v], dp[s | 1 << u][u] + dist[v][u]);
    printf("%d\n", dp[0][0]);
}
```

4. 例题 2

这一题是 TSP 的变形。

hdu 3001 “Travelling”

Acmer 先生决定访问 n 座城市。他可以空降到任意城市,然后开始访问,要求访问到所有城市,任何一个城市访问的次数不少于 1 次,不多于 2 次。 n 座城市间有 m 条道路,每条道路都有路费。求 Acmer 先生完成旅行需要花费的最小费用。

输入:第 1 行是 n 和 m , $1 \leq n \leq 10$; 后面有 m 行,有 3 个整数 a, b, c , 表示城市 a 和 b 之间的路费是 c 。

输出:最少花费,如果不能完成旅行,则输出 -1。

^① 编程细节参考《挑战程序设计竞赛》(秋叶拓哉)192 页,3.4.1 节。

本题 $n=10$, 数据很小, 但是由于每个城市可以走两遍, 可能的路线就变成了 $(2n)!$, 所以不能用暴力法。

本题用状态压缩 DP 求解, 算法复杂度是 $O(3^n n^2)$, 当 $n=10$ 时正好通过 OJ 测试。

1) 路径的表示

在普通 TSP 中, 一个城市只有两种情况, 即访问和不访问, 用 1 和 0 表示。这个题有 3 种情况, 也就是不访问、访问 1 次、访问 2 次, 所以需要用到三进制。

当 $n=10$ 时有 3^{10} 种组合(路径数量), 对每种路径用三进制表示。例如, 第 14 种路径, 它的三进制是 112_3 , 表示第 3 个城市走 1 次, 第 2 个城市走 1 次, 第 1 个城市走 2 次。

在程序中用 $\text{tri}[i][j]$ 表示第 i 个路径, 其第 j 位的值是城市状态, 例如 $\text{tri}[14][3]=1$, $\text{tri}[14][2]=1$, $\text{tri}[14][1]=2$ 。

2) 状态和状态转移

定义状态 $\text{dp}[i][j]$, 当前所在城市是 i , $\text{dp}[i][j]$ 表示从 i 出发访问剩余的所有城市最后回到起点的路径费用总和的最小值。

状态转移: $\text{dp}[j][i] = \min(\text{dp}[j][i], \text{dp}[k][l] + \text{graph}[k][j]);$

```
#include <bits/stdc++.h>
const int INF = 0x3f3f3f3f;
using namespace std;
int n, m;
int bit[12] = {0, 1, 3, 9, 27, 81, 243, 729, 2187, 6561, 19683, 59049};
//三进制每一位的权值, 与二进制的 0、1、2、4、8 等对照

int tri[60000][11];
int dp[11][60000];
int graph[11][11];
void make_trb() {
    for(int i = 0; i < 59050; ++i) {
        int t = i;
        for(int j = 1; j <= 10; ++j) {tri[i][j] = t % 3; t /= 3;}
    }
}
int comp_dp() {
    int ans = INF;
    memset(dp, INF, sizeof(dp));
    for(int i = 0; i <= n; i++)
        dp[i][bit[i]] = 0;
    for(int i = 0; i < bit[n+1]; i++) {
        int flag = 1;
        for(int j = 1; j <= n; j++) {
            if(tri[i][j] == 0) {
                flag = 0;
                continue;
            }
        }
        if(i == j) continue;
        for(int k = 1; k <= n; k++) {
            int l = i - bit[j];
            if(tri[i][k] == 0) continue;
```



```

        dp[j][i] = min(dp[j][i], dp[k][1] + graph[k][j]);
    }
}
if(flag) //找最小费用
    for(int j = 1; j <= n; j++)
        ans = min(ans, dp[j][i]);
}
return ans;
}
int main(){
    make_trb();
    while(cin >> n >> m){
        memset(graph, INF, sizeof(graph));
        while(m--){
            int a, b, c;
            cin >> a >> b >> c;
            if(c < graph[a][b]) graph[a][b] = graph[b][a] = c;
        }
        int ans = comp_dp();
        if(ans == INF) cout << "-1" << endl;
        else cout << ans << endl;
    }
    return 0;
}

```

【习题】

hdu 1074 “Doing Homework”, 入门题。

hdu 2167 “Pebbles”。

hdu 3182 “Hamburger Magi”。

hdu 4539 “排兵布阵”。

poj 1185 “炮兵阵地”, 经典题。

poj 2411 “Mondriaan's Dream”, 铺砖问题。

hdu 3681 “Prison Break”, TSP+二分, 难题。

7.7 小 结

本章介绍了常见的 DP 算法。读者已经看到, DP 题目不仅涉及大量知识点, 而且思维灵活, 不容易掌握。DP 题目作为竞赛的必考题型, 参赛者需要花大量时间练习, 掌握其中的诀窍。

另外还有很多可用 DP 的算法在本章没有涉及, 例如用 DP 解决以概率为最优解的问题, 具体内容见本书 8.4 节; 还有 AC 自动机+DP、后缀自动机+DP 等。